Projet UL2IN002 – 2026fev Thème : “la Mer”

Numéro de groupe : 5

Nom : Kankizil	*Nom :* De Araujo
Prénom : Mikail	*Prénom :* Maxime
N° étudiant : 21129353	*N° étudiant :* 21231531

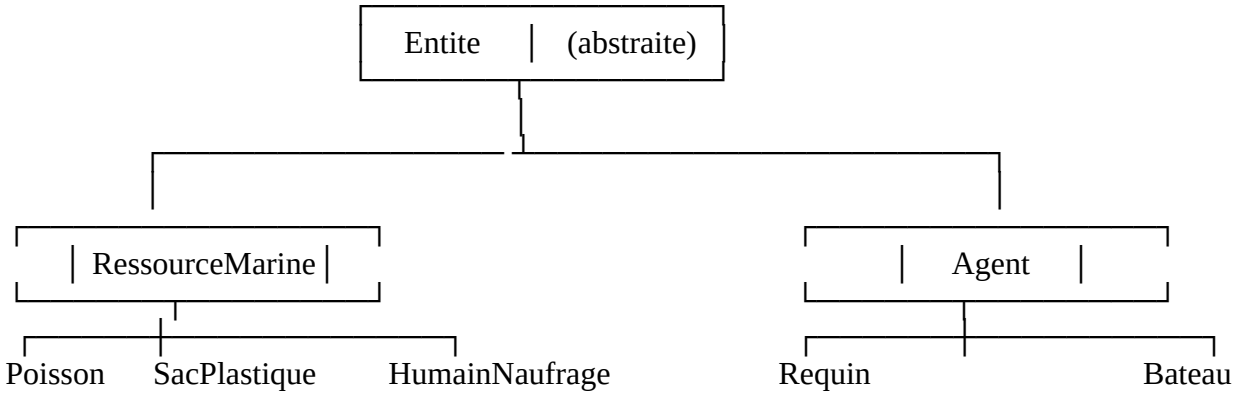
	Thème de la simulation
Thème et objectif(s) de votre simultation : (5 lignes maximum)	Ce projet implémente une simulation d'écosystème marin où des agents interagissent avec des ressources marines sur une grille. La simulation évolue par étapes, suivant des règles d'énergie, de reproduction et d'interactions.
Les agents :	requins et bateaux
Les ressources :	poissons, sacs de plastique, humains naufragés

Quel est le nombre de classes/interfaces de votre projet ? (ne pas compter Ressource, Terrain et TestTerrain)	11 classes et 1 interface
Avez-vous une hiérarchie d'héritage à 3 niveaux ?	Oui
Avez-vous généré la javadoc ?	La documentation Javadoc est disponible dans le dossier `doc/`. Ouvrez `doc/index.html` dans un navigateur pour consulter les détails des classes et méthodes.
Avez-vous généré des logs ?	Oui

Checklist des points vus en cours à mettre en application :	***Nom de classe correspondant (une seule classe demandée, mais vous pouvez en mettre plus)***
Classe contenant une ArrayList	Simulation.java
Interfaces	Entite.java
Classes et méthodes abstraites	Agent.java RessourceMarine.java
Classe avec un constructeur par copie ou méthode clone()	InitialiserAgents() dans Requin.java
Classe contenant que des attributs et méthodes statiques	Utils.java
Classe héritant de Exception	Simulation.java
Gestion des exceptions	Simulation.java

Singleton ou une de ses variantes	SimulationManager.java

Schéma UML des classes vision fournisseur (dessin “à la main” scanné ou photo acceptés)



Copier-coller de 2 ou 3 logs (affichages de simulations)

```
javac *.java
java Simulation
=== Stratégie des bateaux ===
1. PROTECTEUR - Les bateaux sauvent en priorité les naufragés
2. PECHEUR - Les bateaux pêchent en priorité les poissons
3. CHASSEUR - Les bateaux chassent en priorité les requins
Choisissez (1, 2 ou 3, ou tapez 'protecteur', 'pecheur' ou 'chasseur') : 2
=> Mode PÈCHEUR : Les bateaux en quête de poissons !

Les 29 ressources ont les bonnes coordonnées sur le terrain
=== Lancement via SimulationManager ===

--- Étape 0 ---
+------+------+------+------+------+------+------+------+------+------+
|Po    |      |HN    |      |      |HN    |      |HN    |      |      |
+------+------+------+------+------+------+------+------+------+------+
|      |      |      |Po    |      |      |      |      |      |      |
+------+------+------+------+------+------+------+------+------+------+
|Bat   |Po    |Po    |      |      |      |SP    |      |Po    |      |
+------+------+------+------+------+------+------+------+------+------+
|      |      |      |      |      |      |      |      |Po    |      |
+------+------+------+------+------+------+------+------+------+------+
|HN    |SP    |      |      |      |      |      |      |      |      |
+------+------+------+------+------+------+------+------+------+------+
|      |      |      |      |      |Po    |HN    |      |Po    |Rq    |
+------+------+------+------+------+------+------+------+------+------+
|Rq    |HN    |Po    |      |      |      |      |      |Po    |      |
+------+------+------+------+------+------+------+------+------+------+
|      |Po    |      |      |      |      |      |SP    |      |      |
+------+------+------+------+------+------+------+------+------+------+
|Po    |      |Po    |Po    |      |      |Po    |Po    |      |      |
+------+------+------+------+------+------+------+------+------+------+
|      |Po    |      |      |      |Po    |Po    |      |      |Po    |
+------+------+------+------+------+------+------+------+------+------+
```

Agents en vie :
- Requin(7,1) énergie=16
- Requin(6,10) énergie=16
- Bateau(3,1) énergie=50
Humains sauvés : 0 | Humains mangés : 0 | Poissons pêchés : 0 | Poissons perdus : 0

--- Étape 25 ---

```
+------+------+------+------+------+------+------+------+------+------+
|Po    |Po    |      |      |      |      |      |      |Po    |      |
+------+------+------+------+------+------+------+------+------+------+
|      |      |      |      |      |      |      |      |      |      |
+------+------+------+------+------+------+------+------+------+------+
|      |      |      |      |      |      |SP    |      |      |      |
+------+------+------+------+------+------+------+------+------+------+
|      |      |      |      |      |      |      |      |      |      |
+------+------+------+------+------+------+------+------+------+------+
|HN    |SP    |      |      |      |      |      |      |      |      |
+------+------+------+------+------+------+------+------+------+------+
|      |      |      |      |      |      |      |      |      |      |
+------+------+------+------+------+------+------+------+------+------+
|      |      |      |      |      |      |      |      |      |Po    |
+------+------+------+------+------+------+------+------+------+------+
|      |      |      |      |      |      |      |SP    |      |      |
+------+------+------+------+------+------+------+------+------+------+
|      |Rq    |      |      |      |      |      |      |      |      |
+------+------+------+------+------+------+------+------+------+------+
|      |Po    |      |Bat   |      |Rq    |      |      |      |      |
+------+------+------+------+------+------+------+------+------+------+
```

Agents en vie :
- Requin(9,2) énergie=6
- Requin(10,6) énergie=10
- Bateau(10,4) énergie=44
Humains sauvés : 1 | Humains mangés : 4 | Poissons pêchés : 83 | Poissons perdus : 0

--- Bilan final ---
Agents restants : 3
Humains sauvés : 1
Humains mangés : 4
Poissons pêchés : 83
Poissons perdus : 0
Ressources restantes : 9
Bilan mitigé : un équilibre fragile entre sauvetage, prédation, pêche et pertes.
Temps d'exécution : 52090 ms

Copier-coller vos classes et interfaces à partir d'ici :

Classes :
Agent.java
Bateau.java
HumainNaufragé.java
Poisson.java
Requin.java

RessourceMarine.java
Ressource.java
SacPlastique.java
Simulation.java
SimulationException.java
SimulationManager.java
Terrain.java
TestTerrain.java
Utils.java

Interface :
Entité.java

```
import java.util.List;
import java.util.Random;

/**
* Classe abstraite représentant un agent mobile dans la simulation.
* Les agents ont de l'énergie, une position sur la grille et peuvent se déplacer.
*/
public abstract class Agent implements Entite {
protected static final Random RNG = new Random();
protected final String type;
protected int energie;
protected int ligne;
protected int colonne;
protected final int nbLignes;
protected final int nbColonnes;

/**
* Constructeur pour créer un agent.
* @param type Le type de l'agent (ex: "Requin", "Bateau").
* @param energie L'énergie initiale de l'agent.
* @param ligne La ligne initiale sur la grille.
* @param colonne La colonne initiale sur la grille.
* @param nbLignes Le nombre total de lignes de la grille.
* @param nbColonnes Le nombre total de colonnes de la grille.
*/
public Agent(String type, int energie, int ligne, int colonne, int nbLignes, int nbColonnes) {
this.type = type;
this.energie = energie;
this.ligne = Math.max(1, Math.min(nbLignes, ligne));
this.colonne = Math.max(1, Math.min(nbColonnes, colonne));
this.nbLignes = nbLignes;
this.nbColonnes = nbColonnes;
}

/**
```

```
* Retourne le type de l'agent.
* @return Le type sous forme de chaîne.
*/
@Override
public String getType() {
return type;
}

/**
* Retourne la ligne actuelle de l'agent.
* @return La ligne.
*/
@Override
public int getLigne() {
return ligne;
}

/**
* Retourne la colonne actuelle de l'agent.
* @return La colonne.
*/
@Override
public int getColonne() {
return colonne;
}

/**
* Définit la position de l'agent sur la grille, en s'assurant qu'elle reste dans les limites.
* @param ligne La nouvelle ligne.
* @param colonne La nouvelle colonne.
*/
@Override
public void setPosition(int ligne, int colonne) {
this.ligne = Math.max(1, Math.min(nbLignes, ligne));
this.colonne = Math.max(1, Math.min(nbColonnes, colonne));
}

/**
* Calcule la distance de Manhattan entre la position actuelle et une case donnée.
* @param lig La ligne cible.
* @param col La colonne cible.
* @return La distance.
*/
protected int distance(int lig, int col) {
return Math.abs(ligne - lig) + Math.abs(colonne - col);
}

/**
* Calcule la distance de Manhattan entre cet agent et une autre entité.
* @param other L'autre entité.
```

```
* @return La distance.
*/
protected int distance(Entite other) {
return distance(other.getLigne(), other.getColonne());
}

/**
* Déplace l'agent d'un pas vers la cible donnée.
* @param targetLigne La ligne cible.
* @param targetColonne La colonne cible.
*/
protected void moveTowards(int targetLigne, int targetColonne) {
int dL = Integer.compare(targetLigne, ligne);
int dC = Integer.compare(targetColonne, colonne);
setPosition(ligne + dL, colonne + dC);
}

/**
* Déplace l'agent de manière aléatoire (dans une direction aléatoire ou reste sur place).
*/
protected void moveRandomly() {
int dL = RNG.nextInt(3) - 1;
int dC = RNG.nextInt(3) - 1;
setPosition(ligne + dL, colonne + dC);
}

/**
* Vérifie si l'agent est encore en vie (énergie > 0).
* @return true si vivant, false sinon.
*/
public boolean isAlive() {
return energie > 0;
}

/**
* Force l'agent à se séparer d'un autre en se déplaçant dans une direction aléatoire.
* Utilisé après un combat pour éviter les bagarres continues.
*/
public void separer() {
moveRandomly();
}

/**
* Méthode abstraite définissant l'action de l'agent à chaque tour.
* @param terrain Le terrain de la simulation.
* @param allAgents La liste de tous les agents.
*/
public abstract void agir(Terrain terrain, List<Agent> allAgents);

/**
```

```
* Retourne une description textuelle de l'agent.
* @return La description.
*/
@Override
public String description() {
return type + "(" + ligne + "," + colonne + ") énergie=" + energie;
}

/**
* Retourne la description de l'agent (même que description()).
* @return La chaîne de description.
*/
@Override
public String toString() {
return description();
}
}
```

```
import java.util.List;

/**
* Classe représentant un bateau, agent sauveteur dans la simulation.
* Le bateau cherche les ressources (poissons, humains) et les requins proches pour se déplacer.
*/
public class Bateau extends Agent {
/**
* Énumération des stratégies possibles pour un bateau.
*/
public enum Strategie {
/** Priorité aux naufragés. */
PROTECTEUR,
/** Priorité à la pêche. */
PECHEUR,
/** Priorité à la chasse aux requins. */
CHASSEUR
}

private int humainsSauves = 0;
private int poissonsPris = 0;
private final Strategie strategie;
```

```java
/**
* Constructeur pour créer un bateau avec une stratégie.
* @param vie L'énergie initiale.
* @param ligne La ligne initiale.
* @param colonne La colonne initiale.
* @param nbLignes Nombre total de lignes.
* @param nbColonnes Nombre total de colonnes.
* @param strategie La stratégie du bateau (PROTECTEUR ou PECHEUR).
*/
public Bateau(int vie, int ligne, int colonne, int nbLignes, int nbColonnes, Strategie strategie) {
super("Bateau", vie, ligne, colonne, nbLignes, nbColonnes);
this.strategie = strategie;
}

/**
* Définit l'action du bateau à chaque tour selon sa stratégie.
* @param terrain Le terrain de la simulation.
* @param allAgents La liste de tous les agents.
*/
@Override
public void agir(Terrain terrain, List<Agent> allAgents) {
ResourceMarine humainProche = findNearestHumain(terrain);
ResourceMarine poissonProche = findNearestPoisson(terrain);
Agent cibleRequin = findNearestAgent(allAgents, Requin.class);

if (strategie == Strategie.PROTECTEUR) {
// Mode protecteur : priorité aux naufragés
if (humainProche != null) {
moveTowards(humainProche.getLigne(), humainProche.getColonne());
} else if (poissonProche != null) {
moveTowards(poissonProche.getLigne(), poissonProche.getColonne());
} else {
moveRandomly();
}
} else if (strategie == Strategie.PECHEUR) {
// Mode pêcheur : priorité aux poissons
if (poissonProche != null) {
moveTowards(poissonProche.getLigne(), poissonProche.getColonne());
} else if (humainProche != null) {
moveTowards(humainProche.getLigne(), humainProche.getColonne());
} else {
moveRandomly();
}
} else if (strategie == Strategie.CHASSEUR) {
// Mode chasseur : priorité aux requins
if (cibleRequin != null && distance(cibleRequin) <= 2) {
// Attaque le requin à portée (distance 2)
cibleRequin.energie = Math.max(0, cibleRequin.energie - 5);
} else if (cibleRequin != null) {
moveTowards(cibleRequin.getLigne(), cibleRequin.getColonne());
```

```
} else if (humainProche != null) {
moveTowards(humainProche.getLigne(), humainProche.getColonne());
} else if (poissonProche != null) {
moveTowards(poissonProche.getLigne(), poissonProche.getColonne());
} else {
moveRandomly();
}
}
}

/**
* Retourne la stratégie du bateau.
* @return La stratégie (PROTECTEUR ou PECHEUR).
*/
public Strategie getStrategie() {
return strategie;
}

/**
* Trouve le poisson le plus proche.
* @param terrain Le terrain contenant les ressources.
* @return Le poisson le plus proche, ou null si aucun.
*/
private ResourceMarine findNearestPoisson(Terrain terrain) {
ResourceMarine meilleur = null;
int distanceMin = Integer.MAX_VALUE;
for (Ressource r : terrain.lesRessources()) {
if (r instanceof Poisson) {
ResourceMarine rm = (ResourceMarine) r;
int dist = distance(rm.getLigne(), rm.getColonne());
if (dist < distanceMin) {
distanceMin = dist;
meilleur = rm;
}
}
}
return meilleur;
}

/**
* Trouve le naufragé le plus proche.
* @param terrain Le terrain contenant les ressources.
* @return Le naufragé le plus proche, ou null si aucun.
*/
private ResourceMarine findNearestHumain(Terrain terrain) {
ResourceMarine meilleur = null;
int distanceMin = Integer.MAX_VALUE;
for (Ressource r : terrain.lesRessources()) {
if (r instanceof HumainNaufrage) {
ResourceMarine rm = (ResourceMarine) r;
```

```
int dist = distance(rm.getLigne(), rm.getColonne());
if (dist < distanceMin) {
distanceMin = dist;
meilleur = rm;
}
}
}
return meilleur;
}

/**
* Incrémente le nombre d'humains sauvés par ce bateau.
*/
public void humainSauve() {
humainsSauves++;
}

/**
* Ajoute le nombre de poissons pris par ce bateau.
* @param quantite La quantité de poissons pris.
*/
public void poissonsPrises(int quantite) {
poissonsPris += quantite;
}

/**
* Retourne le nombre de poissons pris par ce bateau.
* @return Le nombre de poissons pris.
*/
public int poissonsPris() {
return poissonsPris;
}

/**
* Retourne le nombre d'humains sauvés par ce bateau.
* @return Le nombre d'humains sauvés.
*/
public int getHumainsSauves() {
return humainsSauves;
}

/**
* Trouve la ressource marine (poisson ou humain) la plus proche.
* @param terrain Le terrain contenant les ressources.
* @return La ressource la plus proche, ou null si aucune.
*/
private ResourceMarine findNearestRessource(Terrain terrain) {
ResourceMarine meilleur = null;
int distanceMin = Integer.MAX_VALUE;
for (Ressource r : terrain.lesRessources()) {
```

```
if (r instanceof Poisson || r instanceof HumainNaufrage) {
ResourceMarine rm = (ResourceMarine) r;
int dist = distance(rm.getLigne(), rm.getColonne());
if (dist < distanceMin) {
distanceMin = dist;
meilleur = rm;
}
}
}
return meilleur;
}

/**
* Trouve l'agent du type spécifié le plus proche.
* @param allAgents La liste de tous les agents.
* @param typeRecherche La classe du type d'agent à rechercher (ex: Requin.class).
* @return L'agent le plus proche du type spécifié, ou null si aucun.
*/
private Agent findNearestAgent(List<Agent> allAgents, Class<? extends Agent> typeRecherche) {
Agent meilleur = null;
int distanceMin = Integer.MAX_VALUE;
for (Agent other : allAgents) {
if (other == this || !typeRecherche.isInstance(other)) {
continue;
}
int dist = distance(other);
if (dist < distanceMin) {
distanceMin = dist;
meilleur = other;
}
}
return meilleur;
}
}
```

```
/**
* Interface pour les entités positionnables sur la grille de la simulation.
* Définit les méthodes de base pour obtenir et définir la position, le type et une description.
*/
public interface Entite {
/**
* Retourne la ligne actuelle de l'entité.
* @return La ligne.
```

```
*/
int getLigne();

/**
* Retourne la colonne actuelle de l'entité.
* @return La colonne.
*/
int getColonne();

/**
* Retourne le type de l'entité.
* @return Le type sous forme de chaîne.
*/
String getType();

/**
* Définit la position de l'entité.
* @param ligne La nouvelle ligne.
* @param colonne La nouvelle colonne.
*/
void setPosition(int ligne, int colonne);

/**
* Retourne une description textuelle de l'entité.
* @return La description.
*/
String description();
}
```

```
/**
* Classe représentant un humain naufragé, ressource marine à sauver.
*/
public class HumainNaufrage extends ResourceMarine {
/**
* Constructeur pour créer un humain naufragé avec quantité 1.
*/
public HumainNaufrage() {
super("HumainNaufrage", 1);
}
}
```

```java
/**
* Classe représentant un poisson, ressource marine qui peut croître.
*/
public class Poisson extends ResourceMarine {
/**
* Constructeur pour créer un poisson.
* @param quantite La quantité initiale de poissons.
*/
public Poisson(int quantite) {
super("Poisson", quantite);
}

/**
* Fait croître le poisson en augmentant sa quantité de 1.
*/
public void croit() {
setQuantite(getQuantite() + 1);
}
}
```

```java
import java.util.List;

/**
* Classe représentant un requin, agent prédateur dans la simulation.
* Le requin cherche les ressources (poissons, humains) et les bateaux proches pour se déplacer.
*/
public class Requin extends Agent implements Cloneable{
/**
* Constructeur pour créer un requin.
* @param vie L'énergie initiale du requin.
* @param ligne La ligne initiale.
* @param colonne La colonne initiale.
* @param nbLignes Nombre total de lignes.
* @param nbColonnes Nombre total de colonnes.
*/
public Requin(int vie, int ligne, int colonne, int nbLignes, int nbColonnes) {
super("Requin", vie, ligne, colonne, nbLignes, nbColonnes);
}

/**
* Définit l'action du requin à chaque tour : se déplacer vers la ressource la plus proche.
* Les requins ne poursuivent pas les bateaux, ils cherchent uniquement la nourriture.
* @param terrain Le terrain de la simulation.
```

```
* @param allAgents La liste de tous les agents.
*/
@Override
public void agir(Terrain terrain, List<Agent> allAgents) {
ResourceMarine cible = findNearestRessource(terrain);
if (cible != null) {
moveTowards(cible.getLigne(), cible.getColonne());
} else {
moveRandomly();
}
energie = Math.max(0, energie);
}

/**
* Trouve la ressource marine (poisson ou humain) la plus proche.
* @param terrain Le terrain contenant les ressources.
* @return La ressource la plus proche, ou null si aucune.
*/
private ResourceMarine findNearestRessource(Terrain terrain) {
ResourceMarine meilleur = null;
int distanceMin = Integer.MAX_VALUE;
for (Ressource r : terrain.lesRessources()) {
if (r instanceof Poisson || r instanceof HumainNaufrage) {
ResourceMarine rm = (ResourceMarine) r;
int dist = distance(rm.getLigne(), rm.getColonne());
if (dist < distanceMin) {
distanceMin = dist;
meilleur = rm;
}
}
}
return meilleur;
}

/**
* Trouve l'agent du type spécifié le plus proche.
* @param allAgents La liste de tous les agents.
* @param typeRecherche La classe du type d'agent à rechercher (ex: Bateau.class).
* @return L'agent le plus proche du type spécifié, ou null si aucun.
*/
private Agent findNearestAgent(List<Agent> allAgents, Class<? extends Agent> typeRecherche) {
Agent meilleur = null;
int distanceMin = Integer.MAX_VALUE;
for (Agent other : allAgents) {
if (other == this || !typeRecherche.isInstance(other)) {
continue;
}
int dist = distance(other);
if (dist < distanceMin) {
distanceMin = dist;
```

```
meilleur = other;
}
}
return meilleur;
}

/**
* Clone le requin avec une position différente pour éviter les doublons au même endroit.
* @return Un clone du requin avec une nouvelle position aléatoire.
* @throws AssertionError si le clonage échoue.
*/
@Override
public Requin clone() {
try {
Requin copie = (Requin) super.clone();
int newLigne;
int newColonne;
do {
newLigne = 1 + Utils.random(nbLignes);
newColonne = 1 + Utils.random(nbColonnes);
} while (newLigne == ligne && newColonne == colonne);
copie.setPosition(newLigne, newColonne);
return copie;
} catch (CloneNotSupportedException e) {
throw new AssertionError("Clonage impossible");
}
}

}
```

```
/**
* Classe abstraite pour les ressources marines, étendant Ressource et implémentant Entite.
* Permet aux ressources d'avoir une position sur la grille.
*/
public abstract class ResourceMarine extends Ressource implements Entite {
/**
* Constructeur pour créer une ressource marine.
* @param type Le type de la ressource (ex: "Poisson").
* @param quantite La quantité initiale.
*/
public ResourceMarine(String type, int quantite) {
super(type, quantite);
}
```

```java
/**
* Retourne la ligne de la ressource.
* @return La ligne.
*/
@Override
public int getLigne() {
return super.getLigne();
}

/**
* Retourne la colonne de la ressource.
* @return La colonne.
*/
@Override
public int getColonne() {
return super.getColonne();
}

/**
* Définit la position de la ressource.
* @param ligne La nouvelle ligne.
* @param colonne La nouvelle colonne.
*/
@Override
public void setPosition(int ligne, int colonne) {
super.setPosition(ligne, colonne);
}

/**
* Retourne le type de la ressource.
* @return Le type.
*/
@Override
public String getType() {
return super.type;
}

/**
* Retourne une description de la ressource.
* @return La description.
*/
@Override
public String description() {
return getType() + "(" + getQuantite() + ") @" + getLigne() + "," + getColonne();
}

/**
* Retourne la description de la ressource.
* @return La chaîne de description.
*/
```

```java
@Override
public String toString() {
return description();
}
}
```

```java
/**
* Classe représentant un sac de plastique, ressource marine nuisible.
*/
public class SacPlastique extends ResourceMarine {
/**
* Constructeur pour créer un sac de plastique avec quantité 1.
*/
public SacPlastique() {
super("SacPlastique", 1);
}
}
```

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Random;
import java.util.Scanner;

/**
* Classe principale de la simulation d'une scène de sauvetage en mer.
* Gère la grille, les agents (requins et bateaux), les ressources marines
* et les interactions entre tous ces éléments.
*/
public class Simulation {
private static final int NB_LIGNES = 10;
private static final int NB_COLONNES = 10;
private static final int NOMBRE_ETAPES = 25;
private static final int TEMPS_PAUSE_MS = 2000;
```

```java
private static final String ANSI_RESET = "\u001B[0m";
private static final String ANSI_RED = "\u001B[31m";
private static final String ANSI_BLUE = "\u001B[34m";
private static final String ANSI_MAGENTA = "\u001B[35m";
private static final Random RNG = new Random();

private final Terrain terrain;
private final List<Agent> agents;
private int humainsSauves = 0;
private int humainsManges = 0;
private int poissonsPeches = 0;
private int poissonsPerdus = 0;
private Bateau.Strategie strategieChoisie;

/**
* Crée et initialise une nouvelle simulation.
* Initialise le terrain, sélectionne la stratégie, place les ressources et crée les agents.
*/
public Simulation() {
terrain = new Terrain(NB_LIGNES, NB_COLONNES);
agents = new ArrayList<>();
try {
selectionnerStrategie();
} catch (SimulationException e) {
System.out.println(e.getMessage());
strategieChoisie = Bateau.Strategie.PROTECTEUR;
}
initialiserRessources();
initialiserAgents();
terrain.verifierPositionRessources();
}

/**
* Point d'entrée de l'application de simulation.
* @param args Paramètres de ligne de commande (non utilisés).
*/
public static void main(String[] args) {
Simulation sim = new Simulation();
SimulationManager.getInstance().lancer(sim);
}

/**
* Demande à l'utilisateur de choisir la stratégie des bateaux.
*
* @throws SimulationException si le choix n'est pas valide.
*/
private void selectionnerStrategie() throws SimulationException {
Scanner scanner = new Scanner(System.in);
System.out.println("=== Stratégie des bateaux ===");
System.out.println("1. PROTECTEUR - Les bateaux sauvent en priorité les naufragés");
```

```
System.out.println("2. PECHEUR - Les bateaux pêchent en priorité les poissons");
System.out.println("3. CHASSEUR - Les bateaux chassent en priorité les requins");
System.out.print("Choisissez (1, 2 ou 3, ou tapez 'protecteur', 'pecheur' ou 'chasseur') : ");

String choix = scanner.nextLine().trim().toLowerCase();
if (choix.equals("1") || choix.equals("protecteur")) {
strategieChoisie = Bateau.Strategie.PROTECTEUR;
System.out.println("=> Mode PROTECTEUR : Les bateaux à sauver les humains !");
} else if (choix.equals("2") || choix.equals("pecheur")) {
strategieChoisie = Bateau.Strategie.PECHEUR;
System.out.println("=> Mode PÈCHEUR : Les bateaux en quête de poissons !");
} else if (choix.equals("3") || choix.equals("chasseur")) {
strategieChoisie = Bateau.Strategie.CHASSEUR;
System.out.println("=> Mode CHASSEUR : Les bateaux en chasse des requins !");
} else {
scanner.close();
throw new SimulationException("Choix invalide, mode PROTECTEUR par défaut.");
}
System.out.println();
scanner.close();
}

/**
* Initialise les ressources marines sur le terrain (poissons, humains, plastiques).
*/
private void initialiserRessources() {
for (int i = 0; i < 20; i++) {
placerRessourceAleatoire(new Poisson(3 + RNG.nextInt(3)));
}
for (int i = 0; i < 3; i++) {
placerRessourceAleatoire(new SacPlastique());
}
for (int i = 0; i < 6; i++) {
placerRessourceAleatoire(new HumainNaufrage());
}
}

/**
* Initialise les agents de la simulation (requins et bateau).
*/
private void initialiserAgents() {
Requin r1 = new Requin(16, 1 + RNG.nextInt(NB_LIGNES), 1 + RNG.nextInt(NB_COLONNES),
NB_LIGNES, NB_COLONNES);
Requin r2 = r1.clone();

agents.add(r1);
agents.add(r2);
agents.add(new Bateau(50, 1 + RNG.nextInt(NB_LIGNES), 1 + RNG.nextInt(NB_COLONNES),
NB_LIGNES, NB_COLONNES, strategieChoisie));
}
```

```java
/**
* Lance la simulation pour un nombre défini d'étapes.
* Affiche l'état initial, puis exécute chaque étape avec un délai.
*/
public void lancer() {
afficherEtat(0);
attendre();
for (int etape = 1; etape <= NOMBRE_ETAPES; etape++) {
mettreAJourLesEtapes(etape);
afficherEtat(etape);
attendre();
}
afficherConclusion();
}

/**
* Met en pause la simulation pendant TEMPS_PAUSE_MS millisecondes.
*/
private void attendre() {
try {
Thread.sleep(TEMPS_PAUSE_MS);
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}
}

/**
* Applique la couleur ANSI appropriée à un agent selon son type.
* @param agent L'agent à coloriser.
* @return La chaîne de l'agent avec codes ANSI de couleur.
*/
private String coloriserAgent(Agent agent) {
if (agent instanceof Requin) {
return ANSI_RED + agent + ANSI_RESET;
}
if (agent instanceof Bateau) {
return ANSI_BLUE + agent + ANSI_RESET;
}
return agent.toString();
}

/**
* Effectue toutes les mises à jour pour une étape de la simulation.
* @param etape Le numéro de l'étape en cours.
*/
private void mettreAJourLesEtapes(int etape) {
try {
if (terrain.lesRessources().isEmpty()) {
throw new SimulationException("Plus aucune ressource !");
```

```
}
} catch (SimulationException e) {
System.out.println(e.getMessage());
return; // ⛔ stop le tour
}
croitreLesPoissons();
List<Agent> snapshot = new ArrayList<>(agents);
for (Agent agent : snapshot) {
if (agent.isAlive()) {
agent.agir(terrain, snapshot);
}
}
resoudreInteractions();
nettoyerMortEtRessources();
gererNouvelleProductionDePoissons();
}

/**
* Fait croître les poissons en augmentant leur quantité.
*/
private void croitreLesPoissons() {
for (Ressource r : terrain.lesRessources()) {
if (r instanceof Poisson) {
((Poisson) r).croit();
}
}
}

/**
* Résout toutes les interactions entre les agents et les ressources de manière cyclique.
*/
private void resoudreInteractions() {
for (Agent agent : new ArrayList<>(agents)) {
if (!agent.isAlive()) {
continue;
}
if (agent instanceof Requin) {
appliquerEffetPlastique((Requin) agent);
mangerRessource((Requin) agent);
}
if (agent instanceof Bateau) {
pecherEtSauver((Bateau) agent);
}
}
resoudreConflitsRequinBateau();
}

/**
* Applique l'effet du sac plastique sur l'énergie d'un requin.
* @param requin Le requin affecté.
```

```
*/
private void appliquerEffetPlastique(Requin requin) {
ResourceMarine ressource = ressourcesSurCase(requin.getLigne(), requin.getColonne());
if (ressource instanceof SacPlastique) {
requin.energie = Math.max(0, requin.energie - 10);
}
}

/**
* Gère la consommation de ressources marines par un requin.
* @param requin Le requin mangeant la ressource.
*/
private void mangerRessource(Requin requin) {
ResourceMarine ressource = ressourcesSurCase(requin.getLigne(), requin.getColonne());
if (ressource instanceof Poisson) {
Poisson poisson = (Poisson) ressource;
int prise = poisson.getQuantite();
poisson.setQuantite(0);
requin.energie += 1;
terrain.viderCase(ressource.getLigne(), ressource.getColonne());
} else if (ressource instanceof HumainNaufrage) {
requin.energie += 2;
humainsManges++;
terrain.viderCase(ressource.getLigne(), ressource.getColonne());
}
}

/**
* Gère la pêche et le sauvetage d'humains par un bateau.
* @param bateau Le bateau effectuant les actions.
*/
private void pecherEtSauver(Bateau bateau) {
ResourceMarine ressource = ressourcesSurCase(bateau.getLigne(), bateau.getColonne());
if (ressource instanceof Poisson) {
Poisson poisson = (Poisson) ressource;
int prise = poisson.getQuantite();
bateau.poissonsPrises(prise);
if (RNG.nextDouble() < 0.05) {
poissonsPeches = Math.max(0, poissonsPeches - 10);
poissonsPerdus += 10;
} else {
poissonsPeches += prise;
}
// Si bateau en mode pêche, donne plus d'énergie
int energieBonus = (bateau.getStrategie() == Bateau.Strategie.PECHEUR) ? (prise + 2) : prise;
bateau.energie = Math.min(50, bateau.energie + energieBonus);
terrain.viderCase(ressource.getLigne(), ressource.getColonne());
} else if (ressource instanceof HumainNaufrage) {
bateau.humainSauve();
humainsSauves++;
```

```
terrain.viderCase(ressource.getLigne(), ressource.getColonne());
}
}

/**
* Résout les conflits entre requins et bateaux utilisant les positions communes.
*/
private void resoudreConflitsRequinBateau() {
for (int i = 0; i < agents.size(); i++) {
for (int j = i + 1; j < agents.size(); j++) {
Agent a = agents.get(i);
Agent b = agents.get(j);
if (a.isAlive() && b.isAlive() && memeCase(a, b)) {
if (a instanceof Requin && b instanceof Bateau) {
b.energie = Math.max(0, b.energie - 2);
a.energie = Math.max(0, a.energie - 3);
a.separer();
b.separer();
} else if (a instanceof Bateau && b instanceof Requin) {
a.energie = Math.max(0, a.energie - 2);
b.energie = Math.max(0, b.energie - 3);
a.separer();
b.separer();
} else if (a instanceof Requin && b instanceof Requin) {
a.energie = Math.max(0, a.energie - 2);
b.energie = Math.max(0, b.energie - 2);
if (a.energie == 0) {
b.energie += 5;
} else if (b.energie == 0) {
a.energie += 5;
}
a.separer();
b.separer();
}
}
}
}
}

/**
* Nettoie les morts et les ressources vides du terrain et de la liste des agents.
*/
private void nettoyerMortEtRessources() {
for (Agent agent : new ArrayList<>(agents)) {
if (!agent.isAlive() && agent instanceof Bateau) {
Bateau bateauMort = (Bateau) agent;
poissonsPerdus = bateauMort.poissonsPris();
humainsSauves -= bateauMort.getHumainsSauves();
humainsManges += bateauMort.getHumainsSauves();
poissonsPeches = Math.max(0, poissonsPeches - poissonsPerdus);
```

```
}
}
agents.removeIf(agent -> !agent.isAlive());
for (Ressource r : new ArrayList<>(terrain.lesRessources())) {
if (r.getQuantite() <= 0) {
terrain.viderCase(r.getLigne(), r.getColonne());
}
}
}

/**
* Vérifie si deux entités occupent la même case sur la grille.
* @param a La première entité.
* @param b La deuxième entité.
* @return true si elles sont sur la même case, false sinon.
*/
private boolean memeCase(Entite a, Entite b) {
return a.getLigne() == b.getLigne() && a.getColonne() == b.getColonne();
}

/**
* Retourne la ressource marine à la position spécifiée, si elle existe.
* @param ligne La ligne de la case.
* @param colonne La colonne de la case.
* @return La ressource marine ou null.
*/
private ResourceMarine ressourcesSurCase(int ligne, int colonne) {
Ressource contenu = terrain.getCase(ligne, colonne);
if (contenu instanceof ResourceMarine) {
return (ResourceMarine) contenu;
}
return null;
}

/**
* Place une ressource aléatoirement sur le terrain en trouvant une case vide.
* @param ressource La ressource à placer.
*/
private void placerRessourceAleatoire(Ressource ressource) {
while (true) {
int lig = 1 + Utils.random(NB_LIGNES);
int col = 1 + Utils.random(NB_COLONNES);
if (terrain.caseEstVide(lig, col)) {
ressource.setPosition(lig, col);
terrain.setCase(lig, col, ressource);
return;
}
}
}
```

```java
/**
* Affiche la grille du terrain avec les agents et ressources colorés.
*/
private void afficherGrilleAgentsEtRessources() {
System.out.println("+------+------+------+------+------+------+------+------+------+------+");
for (int lig = 1; lig <= NB_LIGNES; lig++) {
for (int col = 1; col <= NB_COLONNES; col++) {
System.out.print("|");
System.out.print(coloriserCellule(padderCellule(getCelluleTexte(lig, col)), lig, col));
}
System.out.println("|");
System.out.println("+------+------+------+------+------+------+------+------+------+------+");
}
}

/**
* Formate un contenu de cellule en le complétant avec des espaces.
* @param contenu Le contenu à formater.
* @return Le contenu formaté sur largeur fixe.
*/
private String padderCellule(String contenu) {
int taille = 6;
if (contenu.length() >= taille) {
return contenu.substring(0, taille);
}
StringBuilder sb = new StringBuilder(contenu);
while (sb.length() < taille) {
sb.append(' ');
}
return sb.toString();
}

/**
* Retourne le texte à afficher pour une cellule de la grille.
* @param lig La ligne.
* @param col La colonne.
* @return Le texte de la cellule (agent et/ou ressource).
*/
private String getCelluleTexte(int lig, int col) {
StringBuilder affichage = new StringBuilder();
Ressource ressource = terrain.getCase(lig, col);
if (ressource instanceof ResourceMarine) {
affichage.append(shortResource((ResourceMarine) ressource));
}
List<Agent> agentsCase = agentsSurCase(lig, col);
if (!agentsCase.isEmpty()) {
if (affichage.length() > 0) {
affichage.append('/');
}
affichage.append(shortAgent(agentsCase));
```

```
}
if (affichage.length() == 0) {
return "";
}
return affichage.toString();
}

/**
* Retourne l'abréviation pour afficher une ressource marine.
* @param ressource La ressource marine.
* @return L'abréviation (ex: "Po" pour poisson).
*/
private String shortResource(ResourceMarine ressource) {
if (ressource instanceof Poisson) {
return "Po";
}
if (ressource instanceof SacPlastique) {
return "SP";
}
if (ressource instanceof HumainNaufrage) {
return "HN";
}
return "Poule";
}

/**
* Retourne la liste des agents situés sur une case précise.
* @param lig La ligne.
* @param col La colonne.
* @return Liste des agents à cette position.
*/
private List<Agent> agentsSurCase(int lig, int col) {
List<Agent> result = new ArrayList<>();
for (Agent agent : agents) {
if (agent.getLigne() == lig && agent.getColonne() == col) {
result.add(agent);
}
}
return result;
}

/**
* Applique la couleur ANSI appropriate à une cellule selon les agents présents.
* @param contenu Le contenu de la cellule.
* @param lig La ligne.
* @param col La colonne.
* @return La cellule avec ses codes ANSI de couleur.
*/
private String coloriserCellule(String contenu, int lig, int col) {
List<Agent> agentsCase = agentsSurCase(lig, col);
```

```
if (agentsCase.isEmpty()) {
return contenu;
}
String color;
if (agentsCase.size() == 1) {
Agent agent = agentsCase.get(0);
if (agent instanceof Requin) {
color = ANSI_RED;
} else if (agent instanceof Bateau) {
color = ANSI_BLUE;
} else {
color = ANSI_MAGENTA;
}
} else {
color = ANSI_MAGENTA;
}
return color + contenu + ANSI_RESET;
}

/**
* Retourne l'abréviation pour afficher un ensemble d'agents.
* @param agentsCase La liste des agents sur une case.
* @return L'abréviation (ex: "Bat" pour bateau, "Bagarre3" pour 3 agents).
*/
private String shortAgent(List<Agent> agentsCase) {
if (agentsCase.size() == 1) {
Agent agent = agentsCase.get(0);
if (agent instanceof Requin) {
return "Rq";
}
if (agent instanceof Bateau) {
return "Bat";
}
return "Ag";
}
return "Bagarre" + agentsCase.size();
}

/**
* Ajoute aléatoirement de nouveaux poissons sur la grille pendant la simulation.
* Chaque étape a 50% de chance d'ajouter un groupe de 2-4 poissons.
*/
/**
* Gère la génération aléatoire de nouveaux poissons chaque étape.
* Avec 50% de probabilité, ajoute entre 2 et 4 poissons aléatoirement positionnés.
*/
private void gererNouvelleProductionDePoissons() {
if (Utils.random(100) < 50) {
Poisson poisson = new Poisson(2 + Utils.random(3));
placerRessourceAleatoire(poisson);
```

```
}
}

/**
* Affiche l'état complet de la simulation pour une étape donnée.
* @param etape Le numéro de l'étape à afficher.
*/
private void afficherEtat(int etape) {
System.out.println("\n--- Étape " + etape + " ---");
afficherGrilleAgentsEtRessources();
System.out.println("Agents en vie :");
for (Agent agent : agents) {
System.out.println("- " + coloriserAgent(agent));
}
System.out.println("Humains sauvés : " + humainsSauves + " | Humains mangés : " +
humainsManges + " | Poissons pêchés : " + poissonsPeches + " | Poissons perdus : " +
poissonsPerdus);
}

/**
* Affiche le bilan final de la simulation avec toutes les statistiques.
*/
private void afficherConclusion() {
System.out.println("\n--- Bilan final ---");
System.out.println("Agents restants : " + agents.size());
System.out.println("Humains sauvés : " + humainsSauves);
System.out.println("Humains mangés : " + humainsManges);
System.out.println("Poissons pêchés : " + poissonsPeches);
System.out.println("Poissons perdus : " + poissonsPerdus);
System.out.println("Ressources restantes : " + terrain.compterRessources());
if (humainsSauves > humainsManges && poissonsPeches > poissonsPerdus) {
System.out.println("Bilan positif : les humains ont été majoritairement sauvés et les poissons
pêchés dépassent les pertes.");
} else if (humainsManges > humainsSauves && poissonsPerdus > poissonsPeches) {
System.out.println("Bilan négatif : les humains ont été majoritairement mangés et les pertes de
poissons dépassent les prises.");
} else {
System.out.println("Bilan mitigé : un équilibre fragile entre sauvetage, prédation, pêche et
pertes.");
}
}
}
```

```
/**
```

```
* Exception levée lors d'erreurs dans la simulation.
* Utilisée pour signaler des problèmes de validation ou d'exécution spécifiques à la simulation.
*/
public class SimulationException extends Exception {

/**
* Crée une SimulationException avec un message d'erreur.
* @param message Le message décrivant l'erreur.
*/
public SimulationException(String message) {
super(message);
}
}
```

```
/**
* Gestionnaire de simulation utilisant le pattern Singleton.
* Responsable du lancement et du suivi des simulations, y compris la mesure du temps
d'exécution.
*/
public class SimulationManager {

private static SimulationManager instance;

/**
* Constructeur privé pour empêcher l'instanciation directe (Singleton).
*/
private SimulationManager() {}

/**
* Retourne l'instance unique du SimulationManager (Singleton).
* @return L'instance unique de SimulationManager.
*/
public static SimulationManager getInstance() {
if (instance == null) {
instance = new SimulationManager();
}
return instance;
}

/**
* Lance une simulation et affiche le temps d'exécution.
* @param sim La simulation à lancer.
*/
public void lancer(Simulation sim) {
```

```java
System.out.println("=== Lancement via SimulationManager ===");

long debut = System.currentTimeMillis();

sim.lancer();

long fin = System.currentTimeMillis();

System.out.println("Temps d'exécution : " + (fin - debut) + " ms");
}
}
```

```java
import java.util.ArrayList;

/**
* Exemple d'utilisation des classes <code>Terrain</code>
* et <code>Ressource</code>.
* @author UL2IN002 (2026fev)
* @author Christophe Marsala (2020oct-2022oct)
*/
/**
* Classe de test pour démontrer l'utilisation de la classe Terrain et des ressources.
* Ce programme crée un terrain, place des ressources et affiche l'état du terrain.
*/
public class TestTerrain {
/**
* Point d'entrée du programme de test.
* @param args Arguments de la ligne de commande (non utilisés).
*/
public static void main(String[] args) {

// Exemple de création de terrain
Terrain t = new Terrain(3,7);
// Terrain initial : il est vide
t.afficher(2);

// Informations sur le terrain
System.out.println(t);

// On créé une ressource...
```

```
Ressource ress1 = new Ressource("Thon",5);
System.out.println("\nCréation de ress1="+ress1);
// ... et on la place sur le terrain
if (t.setCase(2,3,ress1))
System.out.println("Ajout de " +ress1+" effectuée !");
else
System.out.println("Ajout de " +ress1+" impossible, la case est occupée !");
// On créé une autre ressource...
Ressource ress2 = new Ressource("Poule",4);
System.out.println("\nCréation de ress2="+ress2);
// ... et on essaie de la placer à la même place
if (t.setCase(2,3,ress2))
System.out.println("Ajout de " +ress2+" effectuée !");
else
System.out.println("Ajout de " +ress2+" impossible, la case est occupée !");
// On essaie de placer la ressource à un autre endroit
if (t.setCase(1,4,ress2))
System.out.println("Ajout de " +ress2+" effectuée !");
else
System.out.println("Ajout de " +ress2+" impossible, la case est occupée !");
// Ajout d'une troisième ressource
if (t.caseEstVide(3,5)) {
t.setCase(3,5,new Ressource("Raie",3));
}
System.out.println();
// Affichage du terrain avec les ressources ajoutées
t.afficher(6);

// Informations sur le terrain
System.out.println(t);
System.out.println();
System.out.println("Contenu d'une case :");
System.out.println("- Contenu de la case (2,3) : "+t.getCase(2,3));
System.out.println("- Contenu de la case (1,4) : "+t.getCase(1,4));
System.out.println("- Contenu de la case (1,1) : "+t.getCase(1,1));

System.out.println();
System.out.println("Vidage d'une case :");
System.out.println("- Vidage de la case (1,4) : "+t.viderCase(1,4));
System.out.println("- Vidage de la case (1,1) : "+t.viderCase(1,1));
System.out.println();
// Affichage du terrain
t.afficher(4);
System.out.println("Liste des ressources présentes actuellement sur le terrain :");
ArrayList<Ressource> liste = t.lesRessources();
for (Ressource r : liste) {
System.out.println("- "+r);
}
}
```

```
}
```

```
/**
* Classe utilitaire fournissant des méthodes mathématiques et de génération aléatoire.
* Offre des fonctions pour générer des nombres aléatoires et limiter les valeurs.
*/
public class Utils {

/**
* Génère un nombre entier aléatoire entre 0 (inclus) et max (exclus).
* @param max La limite supérieure (exclusive).
* @return Un entier aléatoire dans [0, max[.
*/
public static int random(int max) {
return (int)(Math.random() * max);
}

/**
* Limite une valeur entre un minimum et un maximum.
* @param value La valeur à limiter.
* @param min La limite inférieure.
* @param max La limite supérieure.
* @return La valeur limitée dans [min, max].
*/
public static int clamp(int value, int min, int max) {
return Math.max(min, Math.min(max, value));
}
}
```